

Sistema de Reconocimiento de Losetas de Carcassonne

Tabla de Contenidos

1. Instalación
 2. Flujo de Trabajo Completo
 3. Preparación de Datos
 4. Entrenamiento del Modelo
 5. Evaluación
 6. Uso en Producción
 7. Troubleshooting
-

Instalación

Requisitos del Sistema

- Python 3.8+
- CUDA 11.0+ (opcional, para GPU)
- 8GB RAM mínimo
- 2GB espacio en disco

Instalar Dependencias

```
bash

pip install torch torchvision opencv-python numpy pillow
pip install albumentations matplotlib seaborn scikit-learn
pip install tqdm pandas
```

Flujo de Trabajo Completo

Paso 1: Detectar Losetas del Tablero

```
bash
```

```
# Ejecutar el detector de losetas
python carcassonne.py tablero.jpg
```

```
# Esto generará:
# - tiles/tile_000_r0_c0.png
# - tiles/tile_001_r0_c1.png
# - ... (72 losetas individuales)
```

Instrucciones durante la detección:

1. Selecciona 8 losetas de referencia bien distribuidas
2. Usa las esquinas y bordes del tablero
3. Presiona ENTER cuando termines
4. Presiona 's' para guardar las losetas individuales

Paso 2: Preparar Losetas de Referencia

Organiza tus 24 tipos de losetas modelo en un directorio:

```
bash

references/
├── tipo_0.png
├── tipo_1.png
├── ...
└── tipo_23.png
```

Luego prepara las referencias:

```
bash

python annotation_tool.py prepare references/ prepared_references/
```

Paso 3: Anotar Datos

Opción A: GUI (Recomendado)

```
bash

python annotation_tool.py gui tiles/ prepared_references/
```

Controles:

- Selecciona tipo de loseta (0-23)

- Selecciona rotación (0°, 90°, 180°, 270°)
- Marca si tiene ficha de jugador
- Si tiene ficha, indica posición (0-8) y color
- Presiona "Guardar" para cada loseta
- Usa ← → para navegar
- Presiona Ctrl+S para guardar rápido

Opción B: CLI

```
bash  
  
python annotation_tool.py cli tiles/
```

Paso 4: Aumentar el Dataset (Opcional pero Recomendado)

```
bash  
  
# Generar 10 variaciones de cada loseta  
python data_utils.py augment annotations.json augmented_data/ 10  
  
# O generar datos sintéticos  
python data_utils.py synthetic prepared_references/ synthetic_data/ 50
```

Paso 5: Dividir Dataset

```
bash  
  
# División 70% train, 15% val, 15% test  
python data_utils.py split annotations.json 0.7 0.15
```

Esto genera:

- train_annotations.json
- val_annotations.json
- test_annotations.json

Paso 6: Analizar Dataset

```
bash  
  
python data_utils.py analyze train_annotations.json
```

Esto muestra:

- Distribución de tipos
- Balance del dataset
- Distribución de rotaciones
- Estadísticas de fichas

Paso 7: Entrenar Modelo

```
python

# train_model.py
from carcassonne_cnn import *

# Crear datasets
train_dataset = CarcassonneTileDataset(
    'train_annotations.json',
    transform=create_data_transforms(augment=True)
)

val_dataset = CarcassonneTileDataset(
    'val_annotations.json',
    transform=create_data_transforms(augment=False)
)

# Crear dataloaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, num_workers=4)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False, num_workers=4)

# Crear y entrenar modelo
model = CarcassonneCNN()
trainer = CarcassonneTrainer(model)
trainer.train(train_loader, val_loader, epochs=50)
trainer.plot_history()
```

Ejecutar:

```
bash

python train_model.py
```

Paso 8: Evaluar Modelo

```
bash

python evaluate_model.py best_carcassonne_model.pth test_annotations.json
```

Esto genera:

- `evaluation_results/metrics.json`
- `evaluation_results/confusion_matrices.png`
- `evaluation_results/confidence_distribution.png`
- `evaluation_results/error_analysis.json`

Paso 9: Usar en Producción

```
bash
```

```
# Pipeline completo: detección + clasificación
```

```
python pipeline.py best_carcassonne_model.pth nuevo_tablero.jpg
```

Esto genera:

- `board_analysis.png`: Visualización del tablero
- `board_analysis_report.png`: Reporte detallado
- `board_results.json`: Datos completos
- `game_state.json`: Estado del juego estructurado

Preparación de Datos

Formato de Anotaciones

```
json
```

```
[
  {
    "image_path": "tiles/tile_000.png",
    "tile_type": 5,
    "rotation": 1,
    "has_meeple": true,
    "meeple_position": 4,
    "meeple_color": "blue"
  }
]
```

Campos:

- `tile_type`: 0-23 (tipo de loseta según tu catálogo)
- `rotation`: 0=0°, 1=90°, 2=180°, 3=270°

- `has_meeple`: true/false
- `meeple_position`: 0-8 (posición en grid 3x3), -1 si no hay
- `meeple_color`: 'red', 'blue', 'green', 'yellow', 'black', 'none'

Posiciones de Ficha (Grid 3x3)

```
0 1 2
3 4 5
6 7 8
```

- **0**: Superior izquierda
- **1**: Superior centro
- **2**: Superior derecha
- **3**: Centro izquierda
- **4**: Centro (típico para ciudad/carretera)
- **5**: Centro derecha
- **6**: Inferior izquierda
- **7**: Inferior centro
- **8**: Inferior derecha

Entrenamiento del Modelo

Hiperparámetros Recomendados

```
python
```

```
# Para dataset pequeño (<500 imágenes)
```

```
batch_size = 16
```

```
learning_rate = 0.0001
```

```
epochs = 100
```

```
augmentations_per_tile = 15
```

```
# Para dataset mediano (500-2000 imágenes)
```

```
batch_size = 32
```

```
learning_rate = 0.001
```

```
epochs = 50
```

```
augmentations_per_tile = 10
```

```
# Para dataset grande (>2000 imágenes)
```

```
batch_size = 64
```

```
learning_rate = 0.001
```

```
epochs = 30
```

```
augmentations_per_tile = 5
```

Estrategias de Entrenamiento

1. Transfer Learning (Recomendado)

El modelo usa ResNet18 pre-entrenado en ImageNet. Esto es ideal porque:

- Reduce tiempo de entrenamiento
- Necesita menos datos
- Mayor precisión inicial

2. Fine-tuning Gradual

```
python
```

```
# Congelar backbone inicialmente
```

```
for param in model.features.parameters():
```

```
    param.requires_grad = False
```

```
# Entrenar solo los heads (10 epochs)
```

```
trainer.train(train_loader, val_loader, epochs=10)
```

```
# Descongelar todo y entrenar completo
```

```
for param in model.features.parameters():
```

```
    param.requires_grad = True
```

```
trainer.optimizer = optim.AdamW(model.parameters(), lr=0.0001)
```

```
trainer.train(train_loader, val_loader, epochs=40)
```

Monitoreo del Entrenamiento

El trainer guarda automáticamente:

- `best_carcassonne_model.pth`: Mejor modelo según validation loss
- `training_history.png`: Gráficas de loss y accuracy

Señales de buen entrenamiento:

- Train loss y val loss disminuyen juntos
- Tile type accuracy >90%
- Rotation accuracy >85%
- Gap entre train y val <10%

Señales de overfitting:

- Train loss sigue bajando, val loss sube
- Gran diferencia entre train acc y val acc
- **Solución:** Más data augmentation, más regularización

Señales de underfitting:

- Ambos loss altos y estables
- Accuracies bajas (<70%)
- **Solución:** Modelo más grande, más epochs, menos regularización



Evaluación

Métricas Clave

Tipo de Loseta:

- Target: >90% accuracy
- Crítico para el juego

Rotación:

- Target: >85% accuracy
- Importante para colocación correcta

Detección de Ficha:

- Target: >95% accuracy
- Debe ser muy confiable

Posición de Ficha:

- Target: >80% accuracy (solo entre fichas detectadas)
- Menos crítico si la detección es correcta

Interpretar Resultados

```
bash

# Ver métricas detalladas
cat evaluation_results/metrics.json

# Ver errores comunes
cat evaluation_results/error_analysis.json

# Tipos más confundidos
python -c "
import json
with open('evaluation_results/error_analysis.json') as f:
    errors = json.load(f)
    print('Top 5 confusiones:')
# Analizar patrones
"
```

Uso en Producción

Predicción Individual

```
python

from carcassonne_cnn import CarcassonnePredictor

predictor = CarcassonnePredictor('best_carcassonne_model.pth')
result = predictor.predict('loseta.png')

print(f'Tipo: {result['tile_type']}")
print(f'Rotación: {result['rotation']}°")
print(f'Ficha: {result['has_meeple']}")
if result['has_meeple']:
    print(f'Posición: {result['meeple_position']}")
```

Predicción por Lotes

```
python

results = predictor.predict_batch('tiles/')
```

Pipeline Completo

```
python

from pipeline import CarcassonnePipeline

pipeline = CarcassonnePipeline('best_carcassonne_model.pth')
results = pipeline.process_board_image('tablero.jpg')

# Exportar estado del juego
pipeline.export_game_state('partida_actual.json')
```

Integración con Aplicación

```
python

# game_analyzer.py
class GameAnalyzer:
    def __init__(self, model_path):
        self.pipeline = CarcassonnePipeline(model_path)

    def analyze_move(self, board_image):
        """Analiza el estado actual del tablero"""
        results = self.pipeline.process_board_image(board_image)

        return {
            'total_tiles': len(results),
            'player_meeples': self.count_meeples(results),
            'tile_distribution': self.get_tile_distribution(results)
        }

    def count_meeples(self, results):
        meeples = {'red': 0, 'blue': 0, 'green': 0, 'yellow': 0, 'black': 0}
        for r in results:
            if r['has_meeple']:
                # Detectar color (necesitas extender el modelo)
                pass
        return meeples
```

Troubleshooting

Problemas Comunes

1. "CUDA out of memory"

```
python

# Reducir batch size
batch_size = 16 # o incluso 8

# Limpiar caché
torch.cuda.empty_cache()
```

2. "Low accuracy en tipo de loseta"

- Verifica que las anotaciones sean correctas
- Aumenta el dataset (mínimo 50 ejemplos por tipo)
- Usa más data augmentation
- Verifica que las losetas de referencia sean claras

3. "Confusión entre tipos similares"

- Las losetas muy similares se confunden
- Solución: Colectar más ejemplos de esos tipos específicos
- Considerar usar ensemble de modelos

4. "Mala detección de fichas"

- Las fichas pequeñas son difíciles de detectar
- Solución: Asegurar buena calidad de imagen (min 100x100 px por loseta)
- Aumentar ejemplos con fichas en diferentes posiciones

5. "Detección de losetas falla"

- Verifica que las 8 losetas de referencia estén bien distribuidas
- Usa losetas completas y bien visibles como referencia
- Evita losetas en los bordes del tablero como referencia

Optimizaciones

Para Datasets Pequeños

```
python
```

```
# Más augmentation
```

```
augmentations_per_tile = 20
```

```
# Transfer learning fuerte
```

```
for param in model.features.parameters():
```

```
    param.requires_grad = False # Congelar backbone
```

```
# Más regularización
```

```
dropout = 0.5
```

```
weight_decay = 0.01
```

Para Mejor Velocidad

```
python
```

```
# Modelo más pequeño
```

```
backbone = models.resnet18(pretrained=True)
```

```
# Batch size mayor
```

```
batch_size = 64
```

```
# Mixed precision training
```

```
from torch.cuda.amp import autocast, GradScaler
```

```
scaler = GradScaler()
```

Para Mejor Precisión

```
python
```

```
# Modelo más grande
```

```
backbone = models.resnet50(pretrained=True)
```

```
# Ensemble de modelos
```

```
models = [load_model(f'model_{i}.pth') for i in range(3)]
```

```
# Promediar predicciones
```

```
# Test-time augmentation
```

```
predictions = []
```

```
for rotation in [0, 90, 180, 270]:
```

```
    rotated = rotate_image(img, rotation)
```

```
    pred = model(rotated)
```

```
    predictions.append(pred)
```

```
final_pred = average(predictions)
```

Recursos Adicionales

Documentación de APIs

```
python

# Ver ayuda de cualquier clase
from carcassonne_cnn import CarcassonneCNN
help(CarcassonneCNN)

# Ver parámetros del trainer
from carcassonne_cnn import CarcassonneTrainer
help(CarcassonneTrainer.train)
```

Ejemplos de Uso

Ver carpeta `examples/` para scripts completos de ejemplo.

Datasets de Ejemplo

Si no tienes datos propios, puedes generar un dataset sintético:

```
bash

python data_utils.py synthetic prepared_references/ synthetic_dataset/ 100
python data_utils.py split synthetic_dataset/synthetic_annotations.json
```

Tips y Mejores Prácticas

Data Collection

1. **Calidad sobre cantidad:** 100 imágenes bien anotadas > 1000 mal anotadas
2. **Diversidad:** Captura diferentes condiciones de iluminación
3. **Balance:** Asegura ejemplos similares de cada tipo
4. **Verificación:** Revisa anotaciones al menos 2 veces

Training

1. **Start simple:** Comienza con modelo base, luego optimiza
2. **Monitor always:** Revisa training plots cada 10 epochs
3. **Save checkpoints:** Guarda modelos intermedios por si acaso
4. **Validate often:** Evalúa en val set cada epoch

Production

1. **Confidence thresholds:** Filtra predicciones con baja confianza ($<70\%$)
 2. **Human review:** Permite revisión manual de casos dudosos
 3. **Logging:** Registra todas las predicciones para mejorar el modelo
 4. **Versioning:** Mantén track de versiones del modelo en uso
-

✓ Checklist de Producción

Antes de usar el sistema en producción:

- ☐ Accuracy tipo loseta $>90\%$
 - ☐ Accuracy rotación $>85\%$
 - ☐ Accuracy detección ficha $>95\%$
 - ☐ Probado con 10+ tableros reales
 - ☐ Manejo de errores implementado
 - ☐ Logging configurado
 - ☐ Threshold de confianza definido
 - ☐ Plan de actualización del modelo
 - ☐ Documentación de usuario final
-

SOS Soporte

Si encuentras problemas:

1. Revisa esta guía completa
2. Verifica logs de error
3. Prueba con datos sintéticos primero
4. Reduce complejidad (menos clases, menos datos)
5. Verifica versiones de dependencias

Happy Training! 🎮 🎯